

Seven Great Ideas in Architecture

1. Use *abstraction* to simplify design
 - Implementation technology is abstracted to build logical devices
 - Ex. high-level languages abstract details of instruction set architecture
 - Different layers in computers
 - i. Hardware
 - ii. Operating system
 - iii. Drivers
 - iv. Developer code
 2. Make the *common case* fast
 - Select a common benchmark to optimize for
 - Common instructions in instruction sets are selected for optimization
 - Common cases are more likely to occur
 3. Performance via *parallelism*
 - Modern processors have multiple cores and can execute multiple tasks at once
 - If something can be made faster by doing multiple parts at once
 4. Performance via *pipelining*
 - Operations are broken down into stages, which are done on separate hardware
 - i. Allows multiple operations to be implemented simultaneously
 - Every step is doing one thing, so that at the end you have a finished product
 - “Assembly line”
 5. Performance via *prediction*
 - Many values are highly predictable
 - i. We can assume a value and execute on that path until proven otherwise
 - Predicting future events and readying/optimizing for them
 - Ex. frequently opened apps
 6. Hierarchy of *memories*
 - Modern processors have on-chip caches to store frequently used data from memory
 - Organizing information
 - Keeping the most important things closer, for easier access
 7. Dependability via *redundancy*
 - RAID protocol relies on storing multiple copies of data on “inexpensive” disks to protect against data loss from drive failure
 - Backup components/data
 - If something breaks/goes wrong, you have a backup
-

Transistors

- 4 transistors required for AND and OR
- 8 transistors required for XOR

Technology and Limitations

- Latency from long chains of transistors drives a processor's clock cycle
 - The rise of the memory wall means that increases in clock frequency must be matched by the availability of data
 - The fall of Moore's law means that performance increases are no longer driven by advances in production technology
 - The challenges of the power wall mean we've encountered a practical limit in our ability to dissipate energy (ex. heat)
-

Performance Analysis

- The performance of a program on a computing system depends on
 - The size of the program (number of instructions)
 - The rate at which instructions can be executed (cycles or steps)
 - The time required to complete a step (average)
-

Exercise 1.7

Consider the two different implementations of the same instruction set architecture. The instructions can be divided into four classes according to their CPI (classes A, B, C, and D). P1 with a clock rate of 2.5GHz and CPIs of 1, 2, 3 and 3, and P2 with a clock rate of 3GHz and CPIs of 2, 2, 2, 2.

Given a program with a dynamic instruction count of 1,000,000 instructions divided into classes as follows: 10% class A, 20% class B, 50% class C, and 20% class D. Which is faster: P1 or P2?

- a) What is the global CPI for each implementation?

$$P1 = 0.1 \cdot 1 + 0.2 \cdot 2 + 0.5 \cdot 3 + 0.2 \cdot 3 = 2.6 \text{ average CPI}$$

$$P2 = 0.1 \cdot 2 + 0.2 \cdot 2 + 0.5 \cdot 2 + 0.2 \cdot 2 = 2 \text{ average CPI}$$

- b) Find the clock cycles required in both cases.

P1

$$\text{Total cycles needed} = 2.6 \cdot 1,000,000 = 2,600,000$$

$$\text{Cycles/Second} = 2,500,000,000$$

$$\text{Time needed} = \frac{2,600,000}{2,500,000,000} \approx 1.04 \text{ms}$$

P2

$$\text{Total cycles needed} = 2 \cdot 1,000,000 = 2,000,000$$

$$\text{Cycles/Second} = 3,000,000,000$$

Tuesday, September 6, 2025

LEC01 - Course Intro

$$Time\ needed = \frac{2,000,000}{3,000,000,000} \approx 0.667ms$$

Therefore, implementation P2 is faster.